Data Model, Persistence & Integrations

Document 05 of 07

Purpose

Describe the application’s current data ownership, table relationships, persistence boundaries, integrations, migration workflow and operational assumptions. Identify what is implemented, what remains future work, and where the available source evidence is incomplete.

Product	Bulk App - personal bulking assistant
Repository snapshot	GitHub **rd2080/Bulk_App_Personalised**, default branch **main**, reviewed 21 October 2026; local project mirror also inspected.
Status language	**Observed** = documented and supported by repository source; **Future** = absent/unverified; **Deployment-dependent** = depends on target Neon database, configuration or hosted environment.
Title assumption	The package outline naming Document 05 was not available in this task context. This working title follows Document 04’s functional requirements with technical architecture and data ownership.

Executive summary

Current persistence is implemented with Neon PostgreSQL accessed through a shared psycopg connection context manager and repository modules. Versioned SQL migrations document additive schema changes through 008. Data is centered on one current profile, date-keyed check-ins and weight history, food inventory, recipe/ingredient records, and date-keyed meal plans. Voice commands write planned-meal text through existing repositories. Actual intake, reminders, analytics and AI generation are not represented as persisted workflows in the verified source set.

1. System shape and responsibility boundaries

The current documented path is a Streamlit user interface calling feature repositories, with Neon PostgreSQL as the persistent store. Repository modules own query and validation logic; the Neon helper owns connection configuration, commit, rollback and close behavior. Browser speech recognition is a separate client-side integration on the Voice Input page.

Layer	Responsibility	Current evidence
Streamlit pages	Collect profile, inventory, check-in, recipe, plan and reviewed voice-command input; display validation, saved data and outcomes.	app.py; pages/1–4; docs and README.
Repositories/services	Validate domain values, execute SQL, manage record-level transaction scope; CSV service parses/validates pairs.	bulking_app/repositories/*; services/recipe_csv.py.
Connection helper	Read DATABASE_URL from environment or Streamlit secrets; create psycopg connection; commit on success, rollback on exception, close.	bulking_app/neon.py.
Neon PostgreSQL	Persist application records, uniqueness constraints, checks, foreign keys and indexes.	database/migrations/001–008; schema docs.
Browser Web Speech	Recognize speech and return transcript/status to the Streamlit component. Audio handling may involve browser-vendor service.	pages/3_Voice_Input.py; component; voice docs.
AI adapter/request builder	Request context scaffold exists; profile request builder fetches fresh current profile. Provider call or suggestion path not verified.	bulking_app/ai/*; README and request-builder docs.

Repository boundary

- Application SQL should flow through feature repositories and the shared Neon helper; page modules should coordinate user interaction and call domain operations.
- The helper commits after a successful context block and rolls back and re-raises on exception. Related writes inside one helper context share a transaction.
- CSV recipe import is documented as one transaction after validation and explicit user confirmation.
- The local mirror also includes bulking_app/database.py, a SQLite schema/connection module. Current GitHub docs identify Neon as the application database; SQLite helper usage in active page flows was not found in the reviewed current files. Treat it as legacy or unused until maintainers confirm/remove it.

2. Current persisted data model

The following model summarizes migrations and schema docs, not a live database inspection. A migration file in GitHub does not prove that it is applied to a particular Neon branch.

Table	Key fields and constraints	Relationship / owner
food_inventory	id; food_name; availability_type; optional quantity/unit; is_available; created_at/updated_at. Availability in always_available, stock_tracked, on_demand; quantity null or >=0.	Standalone foods; referenced by recipe ingredients.
weight_entries	id; unique entry_date; weight_kg NUMERIC(5,2), constrained 25–350 kg; created_at.	Dated weight history; same date as check-in upsert.
daily_checkins	id; unique checkin_date; optional-compatible weight_kg, sleep_hours, soreness_level, daily_notes; created_at.	One per date. Check-in writes corresponding weight_entries row atomically.
user_profile	Exactly ten required profile fields: age, height_cm, target_weight_kg, diet_type, diet_philosophy, calorie_strategy, cuisine_preference, regional_context, cooking_complexity, meals_per_day.	Single current row enforced by unique expression index on constant true; no id/history.
recipe_library	id; unique recipe_name; type; preference; complexity; total_weight_g; recipe-level calories/protein/carbohydrate/fat.	Parent for recipe_ingredients; recipe-level nutrition authoritative.
recipe_ingredients	recipe_id, food_id composite primary key; positive quantity; required unit.	Recipe FK cascades on recipe delete; food FK restricts food deletion; composition only.
daily_meal_plans	id; unique plan_date; created_at, updated_at.	One plan header per date; parent of planned_meals.
planned_meals	id; plan_id; meal_type; meal_name; description; position; created_at.	Plan FK cascades; list order by position then id; meal content is text.

3. Relationships and write semantics

Key relationships

Current write semantics

- Check-in: validate first, then upsert daily_checkins by date and upsert weight_entries by entry_date in one connection transaction. Re-saving the same date updates rather than adds a duplicate.
- Profile: validate exactly the agreed ten fields and upsert the single profile row. The database uniqueness expression provides concurrency protection against a second row.
- Meal plan: adding creates the date's plan header if absent; meal edits/deletes are scoped to that plan. The schema enforces meal category values, nonblank bounded name and description length, and nonnegative display order.

- Recipe CSV: both CSV files are parsed and validated; ingredient references must resolve; a recipe upsert replaces ingredient composition. The service documents transaction rollback on database error.
- Foreign-key behavior: deleting a recipe cascades to its composition rows; inventory foods referenced by recipes are protected from deletion. Deleting a plan cascades to planned rows.

4. Migration and environment operation

Migrations are numbered SQL files and the repository's versioned source of truth for schema changes. The reviewed series documents: 001 weight entries, 002 daily check-ins, 003 food inventory, 004 check-in weight, 005 widened weight precision and range constraint, 006 profile, 007 recipe library, and 008 daily meal plan.

- Apply migrations to the intended Neon branch/database before deploying code that depends on them. GitHub presence is not deployment. Record which migration versions have actually been applied per environment.
- Migration README says 006–008 are additive and safe to reapply. Migration 005 alters weight column precision and adds a constraint; inspect target schema/data before applying in a new or divergent environment.
- Configure DATABASE_URL as an environment variable locally or Streamlit secret in the hosted app. The helper fails clearly when missing; never commit credentials.
- Local development instructions currently specify Python 3.9+, requirements installation, and Streamlit app launch. Supported runtime/platform specifics should be reconfirmed before deployment.
- No live Neon connection or deployment configuration was inspected for this document. Actual schema parity, credentials, branch, backup policy, and connection limits remain deployment checks.

5. Integrations and data flow

Integration	Data flow	Status / boundary
Neon PostgreSQL	Streamlit/repository → psycopg helper → SQL → Neon	Active documented store. Requires DATABASE_URL; no live connectivity verified.
Browser speech	Microphone → browser Web Speech API → editable transcript → deterministic parser	Transcript is sent into app interaction. App docs say no audio upload to Gemini/OpenAI or separate audio storage; browser vendor may process speech.
Deterministic meal commands	Reviewed text → rule parser → ambiguity result or repository writes → date plan reload	Implemented per GitHub main docs/source. No LLM needed. Local mirror may lag.
AI request preparation	Fresh profile read → structured task/context envelope	Foundation only; no provider invocation, generated suggestion, or saved output verified.
Recipe CSV	Two uploads → strict parsing/reference validation → preview → confirmation → database transaction	Recipe/ingredient import and templates documented. General food/plan export not verified.
Reminders / analytics	No scheduler, delivery service, chart pipeline or actual intake store in verified schema.	Future/unverified. Requires new product/data/operational design.

6. Data governance and known boundaries

- Single-user model: profile has no user id or authentication relation in the reviewed schema. Do not describe multi-user isolation or access control as implemented without additional evidence.
- Profile is current-state only; no profile history or timestamps are documented. Historical analysis must not reconstruct past targets from the current profile.
- Meal plans store free-text meal name/details. No actual-consumption entities, serving quantities, per-item nutrient snapshot, reminders, chart aggregates or suggestion records are present in migrations 001–008.
- Nutrition is stored on recipes at whole-recipe level; recipe ingredients store food composition and quantities only. No ingredient-nutrient rollup is implemented in the reviewed code.

- Voice transcription privacy depends partly on browser behavior. The application docs state that audio is not sent to Gemini/OpenAI and is not separately uploaded/stored by the app, but cannot guarantee browser recognition is on-device.
- No deletion/export contract, retention period, backup/restore runbook, encryption configuration, or user-facing data portability workflow was established in the reviewed source set. Confirm before production promises.

7. Requirements for future data changes

- Add a new immutable migration for each schema evolution; never silently edit an already-applied migration file. Prefer additive changes with explicit backfill and rollback/recovery instructions.
- Keep validation consistent across UI/repository and PostgreSQL constraints. Database constraints are the last line of defense; produce user-readable validation errors.
- Model actual meal logging separately from planned_meals. Proposed fields should cover consumed_at/local date, item identity or free text, quantity and unit, optional plan link, and nutrition provenance; exact schema remains undecided.
- For reminders, persist user consent, schedule, timezone and channel state only after scheduler/delivery ownership is selected. Avoid storing a reminder as delivered before an external provider confirms it.
- For analytics, derive from dated source records with explicit missing-data handling. Do not add cached totals until invalidation and recomputation semantics are defined.
- For AI suggestions, persist the minimum required to reproduce or audit an accepted suggestion; separate generated draft from accepted plan change and capture the input context or version according to privacy policy.

8. Failure handling and operational criteria

Failure case	Required behavior
Missing DATABASE_URL	Show configuration error; do not report saved state.
Database exception during multi-write operation	Rollback all writes in that transaction; surface recoverable error and retain user input.
Constraint violation	Map failure to a useful field or operation message; do not expose credentials or raw connection string.
Missing migration/table/column	Fail clearly as environment/schema mismatch; direct operator to apply reviewed migrations; do not silently create partial schema at page render.
Neon unavailable/timeout	Preserve entered text in session where feasible; offer retry; avoid duplicate mutation on retry (use unique/upsert semantics where appropriate).
CSV import with invalid references	Reject before commit; show row-level validation feedback; no partial recipe or ingredient replacement.
Browser speech unsupported/denied	Allow manual text; do not prevent ordinary plan editing.
Provider/scheduler future outage	Keep core data entry usable; label failed suggestion/reminder delivery and avoid hidden writes.

9. Technical acceptance criteria

These criteria are review targets for maintenance and future delivery; this handover has not run the application or test suite.

D-01 Current With DATABASE_URL configured, repository operations use the shared Neon psycopg helper and close connections after commit/rollback.

D-02 Current A repository exception rolls back the open transaction and propagates an error; no partial writes remain in the transaction.

D-03 Current A check-in save upserts daily_checkins and weight_entries for the same date atomically.

D-04 Current The profile table enforces at most one row and stores exactly the ten documented profile fields.

D-05 Current Recipe composition foreign keys cascade on recipe deletion and restrict deleting a referenced food.

D-06 Current A plan date is unique; plan meal rows are ordered by position then id and constrained to the selected plan on update/delete.

D-07 Current A malformed or unresolved recipe CSV pair produces validation feedback without partially changing persisted recipes/composition.

D-08 Deployment For each environment, deployment checklist records DATABASE_URL source, target Neon branch, and applied migration versions before app release.

D-09 Future Actual consumption records can exist without a plan and never arise solely from creating or editing planned_meals.

D-10 Future Reminder schedules store explicit timezone/consent and delivery result is not recorded as successful until the delivery integration confirms it.

D-11 Future Chart queries expose date range, units, sample count and missing dates; missing values are not synthesized as observations.

D-12 Future AI suggestion generation does not mutate plan/log state before explicit user acceptance; unavailable context/provider leaves existing records unchanged.

10. Source register and verification

Primary GitHub references: rd2080/Bulk_App_Personalised, default branch main - README.md; docs/database/schema.md; docs/database/daily_checkins.md; docs/database/migrations/README.md; database/migrations/001–008; bulking_app/neon.py; feature repositories and services; docs/voice-input.md; and voice/meal-command pages. Local mirror source files were also consulted.

No test suite was run, no Neon database was queried, and no deployed app was exercised. This document describes repository-level evidence and does not certify deployed schema parity or production operations. The PDF was generated, text-checked and rendered for visual inspection.

End of Document 05